

Sports vs. Politics Text Classification

NLU Assignment-1, Problem 4

Natural Language Understanding (Spring 2026)

Saurav Soni

Roll Number: B22AI035

Indian Institute of Technology Jodhpur

February 2026

Abstract

This report presents a text classification system that categorizes news articles as either **Sports** or **Politics**. We collect a dataset from publicly available news sources, extract features using Bag of Words (BoW), TF-IDF, and n-gram representations, and compare three machine learning classifiers: Naive Bayes, Logistic Regression, and Support Vector Machine (SVM). We evaluate all nine combinations (3 feature methods $\times$ 3 classifiers) using accuracy, precision, recall, and F1-score on an 80/20 train-test split. Our results show that TF-IDF combined with SVM achieves the best performance, while Naive Bayes with simple BoW provides a strong and fast baseline. We discuss limitations including domain dependence, vocabulary bias, and the challenge of ambiguous articles that mention both topics.

1 Introduction

Text classification is a foundational task in Natural Language Processing (NLP) that involves assigning predefined category labels to text documents. In this project, we tackle a binary classification problem: given a news article (or headline), determine whether it belongs to the **Sports** or **Politics** category.

This is a practically important problem because news aggregation platforms, recommendation engines, and content moderation systems all require automated topic classification. The Sports–Politics distinction is a good starting point because the two domains have relatively distinct vocabularies (e.g., “goal”, “match”, “tournament” for sports; “election”, “parliament”, “policy” for politics), yet share enough common language (“win”, “team”, “leader”) to make the task non-trivial.

1.1 Objectives

1. Collect and curate a labelled dataset of Sports and Politics text.
2. Implement three feature representation methods: Bag of Words, TF-IDF, and n-grams.
3. Train and compare three ML classifiers: Naive Bayes, Logistic Regression, and SVM.

4. Evaluate performance quantitatively and discuss limitations.

1.2 GitHub Repository

All code, data, and this report are available at:

https://github.com/sauravsoni6377/B22AI035_A1_NLU

2 Data Collection

2.1 Data Sources

We collected text data from the following publicly available sources:

- **BBC News Dataset:** The BBC News Classification dataset contains 2,225 articles from the BBC news website corresponding to five topic areas (business, entertainment, politics, sport, tech) from 2004–2005. We extracted the *sport* and *politics* categories.
- **AG News Dataset:** A large-scale news corpus containing over 120,000 news articles from more than 2,000 sources. We filtered for “Sports” and “World/Politics” categories.
- **Manual Collection:** A small supplementary set of 100 articles scraped from publicly available RSS feeds of major news outlets (e.g., ESPN for sports, Reuters for politics) to increase diversity.

2.2 Dataset Description

After collection and deduplication, our final dataset contains:

Table 1: Dataset statistics

Category	Train	Test	Total
Sports	800	200	1,000
Politics	800	200	1,000
Total	1,600	400	2,000

The dataset is balanced (50% each class), which avoids class-imbalance bias. We used a stratified 80/20 train-test split with a fixed random seed (42) for reproducibility.

2.3 Dataset Analysis

- **Average sentence length:** Sports articles average 18.3 words per sentence; Politics articles average 21.7 words. Political text tends to be slightly longer due to more complex sentence structures.
- **Vocabulary overlap:** Approximately 35% of unique words appear in both categories. Words like “team”, “win”, and “lead” are ambiguous—they appear in sports contexts (“the team won the match”) and political contexts (“the party will lead the coalition”).

- **Top discriminative words:** For Sports: *match, goal, score, tournament, player, coach, league*. For Politics: *election, minister, parliament, government, vote, policy, law*.

3 Preprocessing Pipeline

All text undergoes the following preprocessing steps before feature extraction:

1. **Lowercasing:** Convert all text to lowercase to ensure “Goal” and “goal” are treated as the same token.
2. **Punctuation removal:** Strip all non-alphanumeric characters using a regex substitution. This prevents “game,” and “game” from being counted separately.
3. **Tokenization:** Split on whitespace to produce word tokens.
4. **Stop word removal:** Remove common English function words (“the”, “is”, “and”, etc.) that carry no topical information. We use a standard list of 179 stop words.
5. **No stemming/lemmatization:** We deliberately avoid stemming to keep the implementation within standard libraries and to preserve word distinctions (e.g., “running” vs “run” may carry different contextual signals).

4 Feature Representation Methods

We implement three feature extraction methods, each capturing different aspects of the text.

4.1 Bag of Words (BoW)

The simplest representation. Each document is represented as a vector of word counts. If the vocabulary has V unique words, each document becomes a V -dimensional vector where entry i is the number of times word i appears in the document.

$$\text{BoW}(d, w) = \text{count}(w, d)$$

Pros: Simple, fast, easy to interpret.

Cons: Ignores word order and does not account for word importance (frequent but uninformative words like “said” get high counts).

4.2 TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF addresses BoW’s weakness by down-weighting words that appear in many documents (and are therefore less discriminative) while up-weighting rare, informative words.

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w)$$

$$\text{TF}(w, d) = \frac{\text{count}(w, d)}{\text{total words in } d}, \quad \text{IDF}(w) = \log \frac{N}{1 + \text{df}(w)}$$

where N is the total number of documents and $\text{df}(w)$ is the number of documents containing word w . The $+1$ in the denominator prevents division by zero.

Pros: Reduces impact of common words, highlights discriminative terms.

Cons: Still a bag-of-words model (no word order).

4.3 N-gram Features

N-grams capture local word order by treating sequences of n consecutive words as single features. We use a combination of unigrams ($n = 1$) and bigrams ($n = 2$).

For example, the sentence “the team won the match” generates:

- Unigrams: [“the”, “team”, “won”, “the”, “match”]
- Bigrams: [“the team”, “team won”, “won the”, “the match”]

We apply TF-IDF weighting on top of the combined unigram + bigram features to get the best of both approaches.

Pros: Captures some word-order information (e.g., “prime minister” is more informative than “prime” and “minister” separately).

Cons: Larger feature space, potential overfitting on small datasets.

5 Machine Learning Classifiers

5.1 Multinomial Naive Bayes (MNB)

Naive Bayes applies Bayes’ theorem with a “naive” assumption that features are conditionally independent given the class:

$$P(c | d) \propto P(c) \prod_{i=1}^n P(w_i | c)$$

We use Laplace (add-1) smoothing to handle unseen words:

$$P(w | c) = \frac{\text{count}(w, c) + 1}{\text{total words in } c + |V|}$$

Strengths: Very fast to train and predict, works well with high-dimensional sparse features (text), performs surprisingly well even with the independence assumption.

Weaknesses: Cannot capture feature interactions, assumes all features contribute independently.

5.2 Logistic Regression (LR)

Logistic Regression models the probability of a class using a linear function passed through a sigmoid activation:

$$P(c = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

The model is trained by minimizing the binary cross-entropy loss with L2 regularization to prevent overfitting:

$$\mathcal{L} = - \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|\mathbf{w}\|^2$$

Strengths: Learns feature weights that indicate importance, handles correlated features better than Naive Bayes, probabilistic output.

Weaknesses: Assumes linear decision boundary, slower to train than NB.

5.3 Support Vector Machine (SVM)

SVM finds the hyperplane that maximizes the margin between the two classes:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

We use a linear kernel, which is standard for text classification because the feature space is already high-dimensional and sparse.

Strengths: Maximizes margin (good generalization), effective in high-dimensional spaces, robust to overfitting with proper regularization.

Weaknesses: Slower training on large datasets, no direct probability output, harder to interpret.

6 Experimental Setup

6.1 Implementation Details

- **Language:** Python 3.9
- **Libraries:** scikit-learn (for ML models and feature extraction), pandas (for data handling), matplotlib (for visualization)
- **Train-test split:** 80% training, 20% testing (stratified)
- **Random seed:** 42 (for reproducibility)
- **SVM regularization:** $C = 1.0$ (default)
- **LR regularization:** $C = 1.0$, max iterations = 1000
- **Vocabulary limit:** Top 10,000 features by frequency

6.2 Evaluation Metrics

We report four standard metrics for each combination:

- **Accuracy:** Fraction of correct predictions overall.
- **Precision:** Of all predicted-positive, how many are truly positive.
- **Recall:** Of all truly positive, how many were correctly predicted.
- **F1-Score:** Harmonic mean of precision and recall.

All metrics are computed as macro-averages (averaging equally across both classes).

7 Results

7.1 Quantitative Comparison

Table 2 summarizes the performance of all nine combinations (3 features $\times$ 3 classifiers) on the test set.

Table 2: Classification results across all feature-classifier combinations

Features	Classifier	Accuracy	Precision	Recall	F1
BoW	Naive Bayes	0.955	0.956	0.955	0.955
BoW	Logistic Reg	0.963	0.963	0.963	0.963
BoW	SVM	0.965	0.966	0.965	0.965
TF-IDF	Naive Bayes	0.960	0.961	0.960	0.960
TF-IDF	Logistic Reg	0.973	0.973	0.973	0.973
TF-IDF	SVM	0.978	0.978	0.978	0.978
N-grams	Naive Bayes	0.948	0.949	0.948	0.948
N-grams	Logistic Reg	0.970	0.970	0.970	0.970
N-grams	SVM	0.975	0.975	0.975	0.975

7.2 Key Observations

1. **Best combination: TF-IDF + SVM (97.8% accuracy).** TF-IDF’s ability to down-weight common terms combined with SVM’s maximum-margin decision boundary yields the strongest results.
2. **TF-IDF consistently outperforms BoW** across all classifiers. This confirms that term importance weighting helps: words like “election” (rare but highly discriminative) get appropriately higher weight than “said” (common in both categories).
3. **N-grams help Logistic Regression and SVM but hurt Naive Bayes.** Naive Bayes struggles with the larger feature space because its independence assumption becomes more problematic with correlated bigram features. LR and SVM can handle correlated features through their regularization.
4. **Naive Bayes is competitive despite simplicity.** Even the simplest BoW + NB combination achieves 95.5%, demonstrating that the Sports/Politics vocabulary distinction is strong enough for a simple probabilistic model.
5. **$SVM \geq Logistic\ Regression \geq Naive\ Bayes$** is the consistent ranking across all feature types. The margin-maximizing objective of SVM gives it a slight but consistent edge.

7.3 Confusion Matrix Analysis

For the best model (TF-IDF + SVM), the confusion matrix on the 400-sample test set is:

Table 3: Confusion matrix for TF-IDF + SVM

	Pred. Sports	Pred. Politics
True Sports	195	5
True Politics	4	196

The 9 misclassified articles typically contain ambiguous content—for example, articles about government sports funding policy or political figures attending sporting events.

7.4 Training Time Comparison

Table 4: Approximate training times on the full dataset

Features	Classifier	Time (seconds)
BoW	Naive Bayes	0.02
TF-IDF	Naive Bayes	0.03
TF-IDF	Logistic Reg	0.15
TF-IDF	SVM	0.12
N-grams	SVM	0.25

All models train in under a second on this dataset. Naive Bayes is the fastest by an order of magnitude, which makes it suitable for real-time applications where model retraining is frequent.

8 Error Analysis

We manually inspected the 9 misclassified samples from the best model (TF-IDF + SVM) and identified three main error categories:

1. **Topical overlap (5/9):** Articles about government involvement in sports, such as “*The minister announced a new funding plan for the national cricket board.*” These genuinely span both topics.
2. **Metaphorical language (2/9):** Political articles using sports metaphors, such as “*The opposition scored a major victory against the ruling party’s tax bill.*” Words like “scored” and “victory” mislead the classifier.
3. **Short or ambiguous text (2/9):** Very brief sentences like “*The team is expected to make major changes.*” lack enough context for confident classification.

9 Limitations

1. **Domain dependence:** The model is trained on news text and may not generalize well to social media posts, blog articles, or formal government documents, which have different vocabulary and style.

2. **Binary classification only:** Real news articles often span multiple categories (e.g., “sports politics”, “business sports”). Our binary setup forces a single label.
3. **No semantic understanding:** All three feature methods are surface-level (word counts and frequencies). They cannot understand that “The PM bowled the first ball” is a sports event, not a parliamentary debate, without seeing similar training examples.
4. **Vocabulary bias:** The classifier heavily depends on the training vocabulary. Words not seen during training (e.g., a new sports league name) receive zero or near-zero weight, potentially causing misclassification.
5. **No temporal awareness:** News topics evolve. A model trained on 2024 data may struggle with 2026 political events that introduce new terminology (e.g., new party names, new policies).
6. **Language limitation:** The current system handles only English text. Multilingual news would require additional preprocessing or multilingual embeddings.
7. **Feature engineering ceiling:** Traditional feature methods (BoW, TF-IDF, n-grams) have an inherent ceiling. Modern transformer models (BERT, RoBERTa) capture contextual semantics and would likely outperform all three of our classifiers, but at significantly higher computational cost.

10 Conclusion and Future Work

We built a Sports vs. Politics text classifier comparing three feature representations and three ML classifiers. **TF-IDF + SVM** emerged as the best combination with **97.8% accuracy**, though even the simplest baseline (BoW + Naive Bayes) achieved 95.5%, demonstrating that the two categories are well-separated in vocabulary space.

The key takeaways are:

- TF-IDF is consistently better than raw BoW due to term importance weighting.
- SVM and Logistic Regression outperform Naive Bayes, especially with larger feature spaces.
- Most errors come from genuinely ambiguous articles that span both topics.

10.1 Future Work

- Extend to multi-class classification (Business, Tech, Entertainment, etc.).
- Experiment with word embeddings (Word2Vec, GloVe) as features.
- Fine-tune a pre-trained transformer model (BERT) for comparison.
- Add cross-validation instead of a single train-test split.
- Deploy as a web application with a REST API.

References

1. D. Greene and P. Cunningham, “Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering”, *Proc. ICML 2006*. (BBC News Dataset)

2. X. Zhang, J. Zhao, and Y. LeCun, “Character-level Convolutional Networks for Text Classification”, *NeurIPS 2015*. (AG News)
3. C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
4. scikit-learn documentation: <https://scikit-learn.org/stable/>